

CGA NLU 활용 가이드

대상: 봇·대화 설계 운영자, AI/NLU 전문 담당자

이 문서는 CGA 의 **ML**, **Semantic**, **LLM** 엔진을 선택하고, 각 엔진에 맞는 데이터·설정·테스트·품질 개선 방법을 정리합니다.

1. NLU 기본 개념

- 의도: 사용자의 발화가 무엇을 요청하는지 나타내는 분류 단위
- 학습문장: 의도를 학습하거나 검색 기준으로 사용하기 위해 등록하는 사용자 표현
- 개체: 발화에서 추출해야 하는 업무 값이나 명칭
- 사전: 도메인 용어·동의어·사용자 표현을 해석하기 위한 자산
- 임계값: 결과를 사용 가능한 것으로 인정할 최소 기준
- 유사도: 입력과 후보 데이터가 의미상 얼마나 가까운지 나타내는 값
- 답변 엔진: 분류 결과에 따라 정해진 답변 또는 검색·생성 답변을 선택하는 영역

2. 엔진 선택 기준

구분	ML	Semantic	LLM
기본 방식	의도와 학습문장 중심 분류	임베딩과 벡터 검색 중심	LLM 모델과 지시문 기반 처리
준비 데이터	의도, 학습문장, 개체, 사전	의도 또는 검색 지식, 임베딩·Vector DB	의도, 지시문, Provider·모델
적합한 시작점	구분 가능한 업무 의도가 있고 학습문장을 관리할 때	표현이 달라도 의미가 가까운 검색이 필요할 때	LLM 기반 해석·생성과 모델 운영이 필요할 때
주요 검증	정확도, 오분류, 문장 균형	검색 유사도, 임계값, 인덱스 상태	응답 일관성, 지시문 준수, 지연·비용

주요 위험	의도 중복, 데이터 불균형	임베딩 호환성, 인덱스 불일치	모델 변경, 프롬프트 영향, 응답 편차
-------	----------------	---------------------	--------------------------

세부 모델과 지원 상태는 봇 생성 화면에서 선택 가능한 항목을 기준으로 확인합니다.

3. 공통 운영 절차

1. 봇과 버전을 확인합니다.
2. NLU 방식과 모델을 확인합니다.
3. 답변 방식을 확인합니다.
4. 데이터 또는 연결 설정을 준비합니다.
5. 학습·인덱스 생성·모델 적용 작업을 실행합니다.
6. 시뮬레이터에서 대표 발화를 테스트합니다.
7. 분석·평가·대화 이력에서 결과를 확인합니다.
8. 원인을 분류하고 데이터를 수정한 뒤 다시 테스트합니다.

4. 품질 점검 공통 원칙

- 한 학습문장에는 하나의 핵심 의도를 담습니다.
- 비슷한 의도는 서로를 구분할 수 있는 명사·동사·상황을 포함합니다.
- 특정 의도에만 데이터가 과도하게 몰리지 않도록 균형을 확인합니다.
- 도메인 용어는 개체 또는 사전으로 관리할지 먼저 결정합니다.
- 테스트 발화는 학습문장과 동일한 표현만 반복하지 않습니다.
- 분석 결과에서 오분류가 반복되는 의도 쌍을 우선 개선합니다.

4.1 데이터 기준선 만들기

데이터를 수정하기 전에 현재 품질을 기록합니다.

1. 봇 UUID와 버전을 기록합니다.

2. NLU 방식·모델·답변 방식·언어를 기록합니다.
3. 의도별 학습문장 수와 중복 문장을 확인합니다.
4. 대표·변형·경계·예외·회귀 발화로 테스트 세트를 만듭니다.
5. 봇 테스트의 선택 의도, 적용 단계와 Score 를 기록합니다.
6. 평가 화면의 오분류·낮은 Score·유사 의도 충돌을 저장합니다.

기준선이 없으면 데이터 증가, 모델 변경과 임계값 변경 중 무엇이 결과를 바꾸었는지 구분할 수 없습니다.

4.2 학습문장 작성 규칙

참고 NLU 가이드의 원칙을 CGA 운영 데이터에 적용할 때는 다음 순서를 사용합니다.

1. 의도 이름을 업무 목적이 드러나도록 정합니다.
2. 한 학습문장에는 하나의 핵심 요청만 넣습니다.
3. 같은 의도를 서로 다른 어순·말투·표현으로 준비합니다.
4. 다른 의도와 겹치는 단어만 늘리지 말고, 의도를 구분하는 상황과 행동을 포함합니다.
5. 동일하거나 거의 같은 문장을 반복 등록하지 않습니다.
6. 특정 의도에만 문장이 몰리지 않는지 의도별로 비교합니다.

예를 들어 **휴가 잔여일 조회**와 **휴가 신청 방법**을 나눌 때는 두 의도에 모두 **휴가**라는 단어를 넣는 것보다 **남은 일수**와 **신청 절차**처럼 구분 기준이 드러나는 표현을 각각 준비합니다.

4.3 QA-지식 데이터 작성 규칙

QA 나 문서 기반 지식을 준비할 때는 질문과 답변의 범위를 분명히 합니다.

- 질문은 실제 사용자가 입력할 수 있는 표현으로 작성합니다.
- 답변은 질문에 직접 대응하도록 작성하고, 한 답변에 여러 주제를 섞지 않습니다.
- 문서 기반 데이터는 제목·본문의 구분이 유지되도록 원문 구조를 정리합니다.
- 표나 목록이 중요한 경우 변환 후 의미가 유지되는지 확인합니다.
- 문서 개정 시 기존 문서와 새 문서가 동시에 검색되지 않는지 적용 상태를 확인합니다.

4.4 운영 점검 기록

엔진 설정이나 학습 데이터를 변경할 때는 다음 항목을 함께 기록합니다.

- 봇과 버전
- 변경 전·후 NLU 방식, 모델, 답변 방식
- 변경한 의도·개체·사전·QA 또는 연결 설정
- 실행한 학습·인덱싱·적용 작업과 화면 상태
- 대표 성공 발화와 실패 발화
- 변경 전·후 시뮬레이터·분석·평가 결과

이 기록이 없으면 엔진 변경의 영향과 데이터 변경의 영향을 분리하기 어렵습니다.

5. 엔진별 가이드

- ML 엔진 활용
- Semantic 엔진 활용
- LLM 엔진 활용

6. ML 엔진 활용

6.1 설정

현재 화면에서 ML 모델로 확인되는 항목은 DeepLearning Lite, TF-IDF Linear, Keyword Baseline 입니다. 실제 학습 연결 상태는 선택 가능 상태와 버전 설정을 함께 확인합니다.

6.2 데이터 작성

- 의도별로 대표 표현과 다양한 표현을 준비합니다.
- 한 문장에 하나의 의도만 포함합니다.
- 방법 문의와 오류 문의처럼 구분이 필요한 의도는 표현을 섞지 않습니다.
- 의도별 문장 수와 표현의 다양성을 함께 확인합니다.

6.3 테스트와 개선

1. 대표 발화와 경계 발화를 각각 준비합니다.
2. 시뮬레이터에서 결과를 확인합니다.
3. 분석·평가 화면에서 오분류 의도와 반복되는 표현을 찾습니다.
4. 경쟁 의도 간 차이가 드러나도록 데이터를 수정합니다.
5. 다시 학습하고 동일 테스트를 반복합니다.

6.4 주의사항

학습문장 수만 늘리는 방식으로 품질이 자동 개선된다고 가정하지 않습니다. 단순한 어미 변화보다 의도를 구분하는 핵심 표현과 실제 사용자의 다양한 표현을 추가합니다.

ML 변경 전 점검:

1. 경쟁 의도와 구분되는 기준을 한 문장으로 설명합니다.
2. 각 의도에 대표 표현·변형 표현·경계 표현이 있는지 확인합니다.
3. 한 문장에 여러 의도를 섞은 학습문장이 없는지 확인합니다.
4. 변경 전 성공 발화가 변경 후에도 유지되는지 재테스트합니다.

7. Semantic 엔진 활용

7.1 유형

- **Semantic - Vector Worker:** CGA Vector Worker 기본 모델과 Local Vector DB 를 사용하는 유형
- **Semantic - External Embedding:** 외부 임베딩과 Local Vector DB 를 연결하는 유형

7.2 설정

Semantic NLU 를 선택하면 Intent Vector DB 연결 설정이 표시됩니다. External Embedding 유형에서는 검색 API 주소와 API Key 선택 입력, Index 이름이 사용될 수 있습니다. 기본 Vector Worker 유형은 기본 연결과 Index 이름을 확인합니다.

7.3 모델 선택

현재 코드에는 한국어 일반 문서용 **ko-sroberta**, 다국어·표·서식용 **multilingual-e5**, 긴 문서·약관용 **bge-m3** 등 외부 임베딩 선택지가 정의되어 있습니다. 실제 운영 선택은 문서 특성, 운영 연결, 임베딩 호환성을 함께 확인합니다.

7.4 테스트와 개선

1. 대표 질문과 표현을 준비합니다.
2. 의도 또는 지식 데이터가 Vector DB 에 반영되었는지 확인합니다.
3. 시뮬레이터에서 의미가 같은 표현과 다른 표현을 각각 테스트합니다.
4. 검색 결과, 유사도, 임계값, 인덱스 상태를 확인합니다.
5. 검색이 맞지 않으면 데이터·임베딩·인덱스 조합을 함께 점검합니다.

Semantic 변경 전 점검:

1. 임베딩 모델과 검색 대상 데이터가 호환되는지 확인합니다.
2. Index 이름과 연결 대상이 현재 봇·버전과 일치하는지 확인합니다.
3. 외부 검색 API 를 사용하는 경우 응답 규격과 인증 설정을 담당자에게 확인합니다.
4. 인덱스가 갱신되기 전 결과를 새 데이터의 품질로 해석하지 않습니다.

8. LLM 엔진 활용

8.1 설정

LLM Engine 을 선택하면 LLM Provider 와 Provider 별 세부 모델을 설정할 수 있습니다. 화면에서 확인된 Provider 선택지는 Gemini, ChatGPT, Claude, Groq, Cerebras, Mistral, Ollama, OpenRouter 입니다. 예를 들어 ChatGPT 를 선택하면 **GPT-4o mini (기본)**과 **GPT-4o (고품질)**이 표시됩니다. Provider 와 모델 목록은 운영 설정에 따라 달라질 수 있으며, Ollama 를 사용하는 경우 별도 주소 입력 항목이 표시될 수 있습니다.

8.2 지시문과 답변 방식

LLM 은 NLU 모델 선택과 답변 방식 선택을 함께 검토해야 합니다.

- LLM Engine 답변: LLM 이 답변을 생성하는 방식
- LLM Engine RAG 답변: 검색된 지식과 LLM 을 함께 사용하는 방식
- 정해진 답변: 미리 정의된 답변을 사용하는 방식

지시문에는 말투, 답변 형식, 제한사항을 명확하게 기록합니다. 지시문 변경 후에는 동일한 테스트 세트로 일관성과 예외 응답을 비교합니다.

8.3 테스트와 개선

1. 대표 질문, 모호한 질문, 금지 또는 예외 질문을 준비합니다.
2. Provider 와 세부 모델을 고정합니다.
3. 동일 입력을 반복해 응답 일관성을 확인합니다.
4. 지시문 준수 여부와 답변 근거를 확인합니다.
5. 지연·비용·실패 응답을 기록합니다.

LLM 변경 전 점검:

1. Provider 와 세부 모델을 변경 전·후에 기록합니다.
2. 동일한 입력·지시문·답변 방식을 사용해 비교합니다.
3. 사실성, 형식 준수, 금지 응답, 실패 응답을 따로 확인합니다.
4. 지연이나 비용이 중요한 업무라면 품질 결과와 함께 기록합니다.

9. 분석과 품질 개선

분석 화면에서는 누적 분류 결과와 적용된 단계를 확인합니다. 현재 화면에 표시되는 분류 단계에는 제외/무시, 스몰토크, Exacting Matching, 룰, ML, 시맨틱, LLM 등이 포함될 수 있습니다.

품질 개선 순서:

1. 실패 발화를 모읍니다.

2. 실제 적용된 분류 단계를 확인합니다.
3. 의도·검색·지시문·답변 중 어느 영역의 문제인지 분리합니다.
4. 최소 범위의 데이터를 수정합니다.
5. 기존 성공 발화와 실패 발화를 함께 재테스트합니다.

9.1 테스트 세트 구성

엔진을 변경하거나 데이터를 수정할 때는 하나의 테스트 발화만 사용하지 않습니다.

테스트 세트	목적	예시 기준
대표 발화	정상적인 주요 사용 경로 확인	자주 사용되는 표현
변형 발화	표현 변화에 대한 처리 확인	어순·말투·띄어쓰기 변화
경계 발화	경쟁 의도와 구분되는지 확인	비슷한 단어를 포함한 다른 요청
예외 발화	미지원·모호한 요청의 처리 확인	의도에 속하지 않는 질문
회귀 발화	변경 전 성공 결과의 유지 확인	이전에 성공한 발화

변경 전과 변경 후에 같은 세트를 사용해야 엔진 변경이나 데이터 보완의 효과를 비교할 수 있습니다. 테스트 결과는 봇·버전·엔진·모델·답변 방식과 함께 기록합니다.

10. 오류 대응

증상	우선 확인	다음 조치
예상 의도로 분류되지 않음	봇·버전·엔진·데이터 상태	대표·경계 발화와 경쟁 의도를 함께 검토
Semantic 검색 결과가 없음	Vector DB, Index, 임베딩 호환성	연결·인덱스·데이터 반영 상태 확인
LLM 답변이 흔들림	Provider, 모델, 지시문, 입력	동일 테스트 세트로 비교하고 지시문을 좁힘

학습 또는 준비 상태가 완료되지 않음	학습 이력, 오류 메시지, 선택 조합	화면 상태를 기록하고 운영 담당자에게 전달
-------------------------	----------------------	----------------------------

검증용 ML 봇에서 확인된 사례:

- 증상: 학습 요청은 Queue 에 등록됐지만 새로고침 후 **미학습**으로 유지됨
- 시뮬레이터 결과: **의도 미분류, 학습 문장이 없어 의도 분류를 수행할 수 없습니다.**
- 판단: 학습문장 저장 성공과 ML 학습 완료는 별개의 상태이므로, 학습 완료 전에는 분류 품질을 평가하지 않음
- 조치: 학습 이력의 완료 상태를 확인하고, 이력이 없으면 운영 담당자에게 봇·버전·엔진·요청 시각을 전달

DB 나 CLI 를 직접 수정해 상태를 맞추지 않습니다. 화면의 봇·버전·엔진·오류 메시지·발생 시각을 기록해 운영 담당자에게 전달합니다.

11. 학습·인덱싱 실행과 상태 판정

학습 버튼을 누른 시점과 엔진이 사용할 결과가 준비된 시점은 다를 수 있습니다. 요청 메시지만으로 성공을 판단하지 않습니다.

11.1 실행 전 확인

- 현재 봇 UUID 와 버전이 맞는가
- 의도·학습문장·개체·사전·QA 변경이 저장되었는가
- 선택한 엔진과 모델이 화면에서 지원 상태인가
- Semantic 의 Index 와 Vector Worker 연결이 준비되었는가
- LLM Provider 의 운영 설정이 준비되었는가
- 기존 성공 테스트 세트를 보존했는가

11.2 엔진별 처리

엔진	주요 처리	성공 판정
----	-------	-------

ML	학습 데이터 검증, 모델 학습, 모델 파일 반영	학습 이력 성공, 버전의 학습 상태, 봇 테스트 분류
Semantic	지식 또는 의도 임베딩, Vector DB 인덱스 반영	Worker 상태, 인덱스 반영, 검색 결과와 유사도
LLM	Provider-모델-지시문 적용, 필요 시 RAG 검색 연결	실제 모델 응답, 지시문 준수, 오류-지연 기록

ML-Semantic 작업은 데이터와 실행 환경에 따라 수 분 이상 걸릴 수 있습니다. 처리 중에 같은 요청을 반복하지 않고 학습 이력 또는 Queue 이력에서 기존 요청 상태를 먼저 확인합니다.

11.3 실패 판정

다음 중 하나라도 해당하면 학습 성공으로 판단하지 않습니다.

- 버튼 문구만 원래 상태로 돌아오고 성공 이력이 없음
- 버전이 계속 **미학습**으로 표시됨
- 실패 상태인데 오류 원인이 비어 있음
- Semantic 인덱스는 생성됐지만 검색 대상 건수가 0 임
- LLM 호출이 설정 저장까지만 되고 실제 응답이 없음

12. Score 와 임계값 운영

Score 는 엔진마다 의미와 범위가 다를 수 있으므로 서로 다른 엔진의 숫자를 직접 비교하지 않습니다.

12.1 Cut-off 설정 절차

1. 현재 운영 Cut-off 와 유사 의도 기준을 기록합니다.
2. 정답 발화와 오답 발화의 Score 분포를 수집합니다.
3. 정답을 과도하게 거부하지 않으면서 오답을 차단하는 후보값을 정합니다.
4. 경계 문장과 미지원 질문을 포함한 평가 세트로 비교합니다.

5. 한 번에 하나의 기준만 변경합니다.
6. 변경 후 회귀 발화를 다시 실행합니다.

Cut-off 를 낮추면 분류 건수는 늘 수 있지만 오분류가 증가할 수 있고, 높이면 오분류는 줄지만 미분류가 늘 수 있습니다. 응답률만으로 품질 향상이라고 판단하지 않습니다.

12.2 유사 의도 충돌

유사 의도 충돌이 표시되면 다음 순서로 처리합니다.

1. 두 의도의 업무 목적이 실제로 분리되어야 하는지 확인합니다.
2. 동일·유사 학습문장을 찾습니다.
3. 각 의도를 구분하는 행동, 대상, 상태와 시점을 정의합니다.
4. 구분 표현이 드러나는 문장을 추가하고 공통 표현만 있는 문장은 줄입니다.
5. 두 의도와 무관한 반례를 함께 평가합니다.

13. 다국어 NLU 운영

CGA 는 한국어, 영어, 중국어(간체), 일본어, 베트남어, 프랑스어, 독일어 UI 와 봇 언어를 지원합니다. 언어가 바뀌어도 의도 ID, 메시지 ID, 상태 코드와 API 계약 값은 번역하지 않습니다.

13.1 언어별 데이터 원칙

- 한 봇 버전의 기준 언어를 먼저 확인합니다.
- 번역문이 아니라 해당 언어 사용자가 실제로 쓰는 표현을 수집합니다.
- 어순, 존칭, 띄어쓰기, 성·수, 활용과 악센트 차이를 언어별로 검토합니다.
- 고유명사와 상품명은 사전 또는 개체 정책을 통일합니다.
- 다국어 임베딩 모델을 사용해도 언어별 평가 세트는 따로 유지합니다.

13.2 메시지 언어 선택

기본 메시지는 요청 언어를 우선 조회하고, 없으면 봇 언어를 사용합니다. 각 언어 메시지를 등록한 뒤 해당 언어로 봇 테스트를 실행합니다. 번역되지 않은 메시지가 표시되면 메시지 ID, 요청 언어와 봇 언어를 함께 확인합니다.

13.3 다국어 회귀 테스트

구분	언어별 확인
대표 발화	주요 의도가 올바르게 분류되는가
경계 발화	비슷한 의도와 구분되는가
개체	날짜·숫자·고유명사가 올바르게 추출되는가
메시지	요청 언어 또는 봇 언어의 문장이 표시되는가
RAG	해당 언어 지식이 검색되고 근거가 유지되는가
LLM	지시한 언어와 형식으로 답변하는가

14. 엔진 변경 실험 방법

모델이나 Provider 를 바꿀 때는 운영 버전을 바로 덮어쓰지 않고 별도 작업 버전에서 비교합니다.

1. 기존 버전을 복사해 실험 버전을 만듭니다.
2. 데이터와 임계값을 고정한 채 모델만 바꿉니다.
3. 같은 테스트 세트와 정확도·미분류·지연을 비교합니다.
4. 다음 실험에서는 모델을 고정하고 데이터 또는 지시문만 바꿉니다.
5. 결과와 비용·지연·운영 제약을 함께 기록합니다.
6. 합격 기준을 충족한 버전만 운영 후보로 지정합니다.

14.1 엔진별 추가 기록

- ML: 학습 시간, 의도별 문장 수, 정확도, 오분류 쌍
- Semantic: 임베딩 모델, Dimension, Index, 데이터 건수, 검색 시간, Top-k 와 유사도
- LLM: Provider, 모델, 지시문 버전, RAG 사용 여부, 응답 시간, 실패율과 비용 기준

15. 품질 개선 예제

배송 일정 문의와 배송 상태 조회가 자주 충돌하는 경우를 예로 듭니다.

1. 두 의도의 업무 목적을 각각 "예정일 안내"와 "현재 위치·상태 확인"으로 정의합니다.
2. **언제, 도착 예정, 몇 일**은 일정 문의 후보로 분류합니다.
3. **어디, 출고, 배송 중, 도착**은 상태 조회 후보로 분류합니다.
4. **배송 알려줘**처럼 구분 정보가 없는 문장은 경계 발화로 따로 둡니다.
5. 주문번호 개체 추출 여부가 두 의도 판정에 영향을 주는지 확인합니다.
6. 동일 문장이 양쪽 의도에 중복되어 있으면 제거하거나 실제 목적에 맞게 수정합니다.
7. 학습 후 대표·변형·경계·예외·회귀 세트를 모두 다시 실행합니다.
8. 오분류가 줄어도 기존 성공 문장의 Score 가 크게 낮아지지 않았는지 확인합니다.

16. 운영 반영 체크리스트

- [] 봇 UUID·버전·언어가 기록되어 있다.
- [] 변경 전 기준선과 변경 후 결과가 같은 테스트 세트로 비교되었다.
- [] 학습 또는 인덱싱 성공을 이력과 실제 결과로 확인했다.
- [] 오분류·낮은 Score·유사 의도 충돌을 검토했다.
- [] 대표·변형·경계·예외·회귀 발화가 모두 실행되었다.
- [] 다국어 메시지와 개체 추출이 대상 언어별로 확인되었다.
- [] 실제 채널과 대화-API-Queue 이력에서 결과를 확인했다.
- [] 실패 시 되돌릴 이전 운영 버전이 확인되었다.

관련 문서

- CGA 메뉴얼 전체 보기
- CGA Getting Started
- CGA 사용자 설명서
- 엔진 비교표